

DESARROLLO DE APLICACIONES BÁSICO · TEMA 12

Manual paso a paso Web_Sesion12

Construcción completa, archivo por archivo, de un sitio web con acceso a datos mediante Entity Framework Core y LINQ, procedimientos almacenados y AJAX moderno, sobre la base de datos VentasLeon. Diseñado para ir copiando y pegando cada paso.

Contenido

- 1. Crear el proyecto e instalar EF Core
- 2. Crear la base de datos (VentasLeon.sql)
- 3. Configurar la conexión (appsettings.json)
- 4. Configurar el arranque (Program.cs)
- 5. Carpeta Modelos: las entidades
- 6. El DbContext (VentasLeonContext)
- 7. Carpeta Datos: el repositorio (consultas)
- 8. Diseño: Layout, CSS y JavaScript
- 9. Página de inicio (Index)
- 10. Ejemplo 1 · Consulta con LINQ
- 11. Ejemplo 2 · Procedimientos almacenados
- 12. Ejemplo 3 · AJAX moderno
- 13. Probar la aplicación

Cómo usar este manual: sigue las páginas en orden. Cada bloque oscuro es código para copiar y pegar tal cual en el archivo indicado encima (la ruta naranja).

1. Crear el proyecto e instalar EF Core

Paso 1 En Visual Studio 2026: **Crear un proyecto nuevo** → plantilla **ASP.NET Core Web App (Razor Pages)**.

Paso 2 Nombre de la solución y del proyecto: `Web_Sesion12`.

Paso 3 Framework **.NET 10**, deje marcado HTTPS y clic en **Crear**.

Paso 4 Clic derecho en el proyecto → **Administrar paquetes NuGet** → **Examinar** → instale:

```
Microsoft.EntityFrameworkCore.SqlServer
Microsoft.EntityFrameworkCore.Tools
```

Paso 5 Cree estas carpetas dentro del proyecto (clic derecho → Agregar → Nueva carpeta):

`Modelos` `Datos` `wwwroot\css` `wwwroot\js`

Recuerde: la plantilla puede aparecer en inglés aunque su Visual Studio esté en español. Es la correcta.

2. Crear la base de datos

Abra **SQL Server Management Studio**, conéctese a su servidor y ejecute una sola vez el script `VentasLeon.sql` que acompaña a este manual. Crea la BD, los datos de ejemplo y los procedimientos almacenados.

Formato de fecha: el script usa fechas `AAAAMMDD` (ej. `'20250115'`) para que SQL Server las interprete igual aunque esté en español. No las cambie a `'2025-01-15'` o dará error de conversión.

Verifique que cargó bien:

```
SELECT * FROM Tb_Factura;           -- deben salir 5 filas
SELECT * FROM Tb_Detalle_Factura;  -- deben salir 7 filas
```

Clientes de prueba: C001 , C002 , C003 . El C001 tiene facturas pendientes y canceladas, ideal para probar la deuda.

3. Configurar la conexión

Web_Sesion12 \ appsettings.json

Agregue la sección `ConnectionStrings` (deje el resto del archivo como está):

```
{
  "ConnectionStrings": {
    "VentasLeon": "Server=.;Database=VentasLeon;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Ajuste el servidor a SU máquina. En JSON la barra va **doble** (`\\`):

· Express con Windows:

`Server=.\SQLEXPRESS;Database=VentasLeon;Trusted_Connection=True;TrustServerCertificate=True`

· Con usuario y clave: `...;User Id=BA;Password=TU_CLAVE;TrustServerCertificate=True` (sin `Trusted_Connection`).

4. Configurar el arranque (1 de 2)

Reemplace TODO el contenido del archivo:

Web_Sesion12 \ Program.cs

```
using Microsoft.EntityFrameworkCore;
using Web_Sesion12.Datos;
using Web_Sesion12.Modelos;

// Punto de entrada de la aplicacion. Aqui se registran los servicios
// y se arma la tubería de peticiones (middlewares).
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();

// Se registra el DbContext de EF Core con la cadena de conexión de appsettings.json.
// A partir de aquí, cualquier página o clase puede recibir el contexto por inyección.
builder.Services.AddDbContext<VentasLeonContext>(opciones =>
    opciones.UseSqlServer(builder.Configuration.GetConnectionString("VentasLeon")));

// La capa de datos: expone los métodos de consulta (LINQ y procedimientos almacenados).
builder.Services.AddScoped<VentasRepositorio>();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();           // permite servir css y js desde wwwroot

app.UseRouting();
app.UseAuthorization();

app.MapRazorPages();
```

4. Configurar el arranque (2 de 2)

Web_Sesion12 \ Program.cs (continuación)

```
app.Run();
```

Qué hace: registra el `DbContext` con la cadena de conexión y registra la capa de datos. Desde aquí las páginas reciben todo por inyección.

5. Carpeta Modelos · entidades (1 de 3)

Dentro de **Modelos**, agregue una clase por archivo (clic derecho → Agregar → Clase). Cada clase mapea una tabla.

Modelos \ Cliente.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Web_Sesion12.Modelos;

// Entidad que mapea la tabla Tb_Cliente. Generada al estilo "database-first"
// (en un proyecto real la crea el comando Scaffold-DbContext).
[Table("Tb_Cliente")]
public class Cliente
{
    [Key]
    [Column("Cod_cli")]
    public string CodCli { get; set; } = "";

    [Column("Raz_soc_cli")]
    public string RazSocCli { get; set; } = "";

    [Column("Ruc_cli")]
    public string? RucCli { get; set; }

    [Column("Dir_cli")]
    public string? DirCli { get; set; }

    [Column("Tel_cli")]
    public string? TelCli { get; set; }

    [Column("Est_cli")]
    public int EstCli { get; set; }

    // Propiedad de navegacion: las facturas de este cliente
    public ICollection<Factura> Facturas { get; set; } = new List<Factura>();
}
```


5. Carpeta Modelos · entidades (2 de 3)

Modelos \ Vendedor.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Web_Sesion12.Modelos;

[Table("Tb_Vendedor")]
public class Vendedor
{
    [Key]
    [Column("Cod_ven")]
    public string CodVen { get; set; } = "";

    [Column("Nom_ven")]
    public string NomVen { get; set; } = "";

    [Column("Ape_ven")]
    public string ApeVen { get; set; } = "";

    public ICollection<Factura> Facturas { get; set; } = new List<Factura>();
}
```

Modelos \ DetalleFactura.cs

```
using System.ComponentModel.DataAnnotations.Schema;

namespace Web_Sesion12.Modelos;

// La clave es compuesta (Num_fac + Cod_pro); se configura en el DbContext.
[Table("Tb_Detalle_Factura")]
public class DetalleFactura
{
    [Column("Num_fac")]
    public string NumFac { get; set; } = "";

    [Column("Cod_pro")]
    public string CodPro { get; set; } = "";

    [Column("Can_ven")]
    public int CanVen { get; set; }

    [Column("Pre_ven")]
    public decimal PreVen { get; set; }

    public Factura? Factura { get; set; }
}
```

5. Carpeta Modelos · entidades (3 de 3)

Modelos \ Factura.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Web_Sesion12.Modelos;

[Table("Tb_Factura")]
public class Factura
{
    [Key]
    [Column("Num_fac")]
    public string NumFac { get; set; } = "";

    [Column("Fec_fac")]
    public DateTime FecFac { get; set; }

    [Column("Fec_can")]
    public DateTime? FecCan { get; set; }

    [Column("Est_fac")]
    public int EstFac { get; set; } // 1 pendiente, 2 cancelada, 3 anulada

    [Column("Cod_cli")]
    public string CodCli { get; set; } = "";

    [Column("Cod_ven")]
    public string CodVen { get; set; } = "";

    [Column("Por_igv")]
    public decimal PorIgv { get; set; }

    // Propiedades de navegacion
    public Cliente? Cliente { get; set; }
    public Vendedor? Vendedor { get; set; }
    public ICollection<DetalleFactura> Detalles { get; set; } = new List<DetalleFactura>();

    // Propiedad calculada (no esta en la BD): texto del estado para mostrar
    [NotMapped]
    public string EstadoTexto => EstFac switch
    {
        1 => "Pendiente",
        2 => "Cancelada",
        _ => "Anulada"
    };
}
```

5. Carpeta Modelos · FacturaResumen

Modelos \ FacturaResumen.cs

```
namespace Web_Sesion12.Modelos;

// Entidad SIN clave (keyless): no representa una tabla, sino el resultado
// del procedimiento almacenado usp_ListarFacturasClienteFechas.
// Sus propiedades coinciden con las columnas que devuelve el SP.
public class FacturaResumen
{
    public string Num_fac { get; set; } = "";
    public DateTime Fec_fac { get; set; }
    public string Estado { get; set; } = "";
    public decimal Total { get; set; }
}
```

FacturaResumen no es una tabla: es una clase "sin clave" que recibe el resultado del procedimiento almacenado del Ejemplo 2.

6. El DbContext (1 de 2)

Modelos \ VentasLeonContext.cs

```
using Microsoft.EntityFrameworkCore;

namespace Web_Sesion12.Modelos;

// El DbContext es la sesion con la base de datos. Cada DbSet es una tabla
// consultable. En database-first lo genera Scaffold-DbContext; aqui se deja
// escrito a mano para fines didacticos.
public class VentasLeonContext : DbContext
{
    public VentasLeonContext(DbContextOptions<VentasLeonContext> options)
        : base(options) { }

    public DbSet<Cliente> Clientes { get; set; }
    public DbSet<Vendedor> Vendedores { get; set; }
    public DbSet<Factura> Facturas { get; set; }
    public DbSet<DetalleFactura> DetalleFacturas { get; set; }

    // Conjunto sin clave que recibe el resultado del procedimiento almacenado.
    public DbSet<FacturaResumen> FacturasResumen { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Clave compuesta del detalle (Num_fac + Cod_pro)
        modelBuilder.Entity<DetalleFactura>()
            .HasKey(d => new { d.NumFac, d.CodPro });

        // FacturaResumen no tiene clave: solo recibe filas de un SP/consulta
        modelBuilder.Entity<FacturaResumen>().HasNoKey();

        // Relaciones: se indica que las columnas existentes son las llaves foraneas,
        // para que EF Core no invente nombres de columna que no estan en la BD.
        modelBuilder.Entity<Factura>()
            .HasOne(f => f.Cliente).WithMany(c => c.Facturas).HasForeignKey(f => f.CodCli);
    }
}
```

6. El DbContext (2 de 2)

Modelos \ VentasLeonContext.cs (continuación)

```
modelBuilder.Entity<Factura>()
    .HasOne(f => f.Vendedor).WithMany(v => v.Facturas).HasForeignKey(f => f.CodVen);

modelBuilder.Entity<DetalleFactura>()
    .HasOne(d => d.Factura).WithMany(f => f.Detalles).HasForeignKey(d => d.NumFac);
}
```

Las relaciones son obligatorias: sin el bloque `HasForeignKey`, EF Core inventa columnas (ClienteCodCli, etc.) que no existen en la BD y la consulta falla. Aquí le indicamos que Cod_cli, Cod_ven y Num_fac son las llaves foráneas reales.

7. Carpeta Datos · el repositorio (1 de 2)

Esta clase concentra TODAS las consultas. Las páginas nunca tocan la base directamente: siempre llaman a estos métodos.

Datos \ VentasRepositorio.cs

```
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;
using System.Data;
using Web_Sesion12.Modelos;

namespace Web_Sesion12.Datos;

// Capa de datos. Centraliza el acceso a la base de datos mediante EF Core.
// Las paginas NUNCA consultan la BD directamente: siempre llaman a estos metodos.
public class VentasRepositorio
{
    private readonly VentasLeonContext _ctx;

    // El DbContext llega por inyeccion (registrado en Program.cs)
    public VentasRepositorio(VentasLeonContext ctx)
    {
        _ctx = ctx;
    }

    /* ===== EJEMPLO 1: CONSULTAS CON LINQ ===== */

    // Suma los importes de las facturas pendientes (estado 1) de un cliente.
    // Consulta a la base de datos via LINQ to Entities (join + Sum).
    public async Task<decimal> CalcularDeudaClienteLinqAsync(string codigo)
    {
        decimal deuda = await (from f in _ctx.Facturas
                                join d in _ctx.DetalleFacturas on f.NumFac equals d.NumFac
                                where f.CodCli == codigo && f.EstFac == 1
                                select (decimal)(d.CanVen * d.PreVen)).SumAsync();

        return deuda;
    }

    // Lista las facturas de un cliente entre dos fechas (sintaxis de metodo).
    // Include trae el detalle para poder mostrar el total de cada factura.
```

7. Carpeta Datos · el repositorio (2 de 2)

Datos \ VentasRepositorio.cs (continuación)

```
public async Task<List<Factura>> ListarFacturasLinqAsync(string codigo, DateTime ini, DateTime fin)
{
    return await _ctx.Facturas
        .Include(f => f.Detalles)
        .Where(f => f.CodCli == codigo && f.FecFac >= ini && f.FecFac <= fin)
        .OrderBy(f => f.FecFac)
        .ToListAsync();
}

/* ===== EJEMPLO 2: PROCEDIMIENTOS ALMACENADOS EN EF CORE ===== */

// Ejecuta el SP usp_ListarFacturasClienteFechas y mapea el resultado
// a la entidad sin clave FacturaResumen. La interpolacion se convierte
// en parametros, por lo que es segura frente a inyeccion SQL.
public async Task<List<FacturaResumen>> ListarFacturasSpAsync(string codigo, DateTime ini, DateTime
fin)
{
    return await _ctx.FacturasResumen
        .FromSql($"EXEC usp_ListarFacturasClienteFechas {codigo}, {ini}, {fin}")
        .ToListAsync();
}

// Ejecuta el SP usp_DeudaCliente, que devuelve la deuda por un parametro
// de SALIDA (OUTPUT). Asi se leen los parametros de salida en EF Core.
public async Task<decimal> CalcularDeudaClienteSpAsync(string codigo)
{
    var pCodigo = new SqlParameter("@codigo", codigo);
    var pDeuda = new SqlParameter("@vdeuda", SqlDbType.Decimal)
    {
        Precision = 12,
        Scale = 2,
        Direction = ParameterDirection.Output
    };

    await _ctx.Database.ExecuteSqlRawAsync(
        "EXEC usp_DeudaCliente @codigo, @vdeuda OUTPUT", pCodigo, pDeuda);

    return pDeuda.Value == DBNull.Value ? 0 : (decimal)pDeuda.Value;
}

/* ===== EJEMPLO 3: APOYO PARA EL AJAX ===== */

// Busca un cliente por su codigo. Lo usa el handler AJAX para refrescar
// el panel sin recargar la pagina.
public async Task<Cliente?> BuscarClienteAsync(string codigo)
{
    return await _ctx.Clientes
        .FirstOrDefaultAsync(c => c.CodCli == codigo);
}
}
```

8. Diseño · Layout (1 de 2)

Reemplace el contenido del layout (plantilla común de todas las páginas):

Pages \ Shared \ _Layout.cshtml

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - VentasLeon</title>

  <!-- Bootstrap desde CDN -->
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" />
  <!-- Hoja de estilos propia del sitio -->
  <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
</head>
<body>

  <!-- Menu superior -->
  <nav class="navbar navbar-expand-lg navbar-dark vl-navbar">
    <div class="container">
      <a class="navbar-brand fw-bold" asp-page="/Index">Ventas<span class="vl-accent">Leon</span>
</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#menu">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="menu">
        <ul class="navbar-nav me-auto">
          <li class="nav-item"><a class="nav-link" asp-page="/Index">Inicio</a></li>
          <li class="nav-item"><a class="nav-link" asp-page="/Consulta">1 · Consulta LINQ</a>
</li>
          <li class="nav-item"><a class="nav-link" asp-page="/Procedimiento">2 ·
Procedimiento</a></li>
          <li class="nav-item"><a class="nav-link" asp-page="/Ajax">3 · AJAX</a></li>
        </ul>
        <span class="navbar-text vl-stack">EF Core 10 · LINQ · ASP.NET Core</span>
      </div>
    </div>
  </nav>

  <main class="container vl-main">
```


8. Diseño · Layout (2 de 2)

Pages \ Shared \ _Layout.cshtml (continuación)

```
@RenderBody()  
</main>  
  
<footer class="vl-footer">  
  <div class="container">  
    <span>&copy; @DateTime.Now.Year VentasLeon - Desarrollo de Aplicaciones Basico</span>  
  </div>  
</footer>  
  
<!-- jQuery (lo usan Bootstrap y la validacion del cliente) -->  
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>  
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">  
</script>  
<!-- Script general del sitio. Se carga aqui, asi queda disponible en todas las paginas -->  
<script src="~/js/site-demo.js" asp-append-version="true"></script>  
  
@await RenderSectionAsync("Scripts", required: false)  
</body>  
</html>
```

8. Diseño · JavaScript

wwwroot \ js \ site-demo.js

```
// Script general del sitio. Se enlaza una sola vez desde _Layout.cshtml,  
// por lo que sus funciones quedan disponibles en cualquier pagina.  
  
// Da formato de moneda en soles a un numero  
function formatearSoles(monto) {  
    return "S/ " + Number(monto).toFixed(2);  
}  
  
// Convierte una cadena a numero, aceptando que venga vacia  
function aNumero(texto) {  
    var n = parseFloat(texto);  
    return isNaN(n) ? 0 : n;  
}
```

site-demo.js se enlaza una sola vez desde el Layout, por lo que sus funciones (como **formatearSoles**) quedan disponibles en todas las páginas.

9. Página de inicio · vista (1 de 2)

Pages \ Index.cshtml

```
@page
@model IndexModel
@{
    ViewData["Title"] = "Inicio";
}

<div class="vl-hero">
    <h1>VentasLeon</h1>
    <p class="lead">Sesion 12 · Acceso a datos con Entity Framework Core y LINQ, y AJAX en ASP.NET Core.</p>
    <p>Tres ejemplos sobre la misma base de datos: consultas con LINQ, procedimientos almacenados y actualizacion de la pagina sin recargar.</p>
</div>

<div class="row g-4">
    <div class="col-md-4">
        <div class="card vl-card">
            <div class="card-body">
                <span class="num">1</span>
                <h5 class="card-title">Consulta con LINQ</h5>
                <p class="text-muted">Deuda de un cliente y sus facturas entre dos fechas, usando LINQ to Entities sobre EF Core.</p>
                <a class="btn btn-vl btn-sm" asp-page="/Consulta">Abrir ejemplo</a>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card vl-card">
            <div class="card-body">
                <span class="num">2</span>
                <h5 class="card-title">Procedimiento almacenado</h5>
                <p class="text-muted">El mismo resultado, pero ejecutando procedimientos almacenados desde EF Core (FromSql y parametro de salida).</p>
                <a class="btn btn-vl btn-sm" asp-page="/Procedimiento">Abrir ejemplo</a>
            </div>
        </div>
    </div>
</div>
```

9. Página de inicio · vista (2 de 2)

Pages \ Index.cshtml (continuación)

```
<div class="card vl-card">
  <div class="card-body">
    <span class="num">3</span>
    <h5 class="card-title">AJAX moderno</h5>
    <p class="text-muted">Buscar un cliente "en vivo" y refrescar solo el panel, sin
recargar la pagina (fetch + handler JSON).</p>
    <a class="btn btn-vl btn-sm" asp-page="/Ajax">Abrir ejemplo</a>
  </div>
</div>
</div>
</div>

<div class="vl-nota">
  Antes de probar, ejecute una sola vez el script <strong>VentasLeon.sql</strong> en SQL Server
  para crear la base de datos, los datos de ejemplo y los procedimientos almacenados.
</div>
```

9. Página de inicio · código

Pages \ Index.cshtml.cs

```
using Microsoft.AspNetCore.Mvc.RazorPages;  
  
namespace Web_Sesion12.Pages;  
  
public class IndexModel : PageModel  
{  
    public void OnGet() { }  
}
```

10. Ejemplo 1 · Consulta LINQ (vista) (1 de 2)

Agregue una nueva Razor Page llamada **Consulta** (clic derecho en Pages → Agregar → Razor Page) y reemplace su contenido:

Pages \ Consulta.cshtml

```
@page
@model ConsultaModel
@{
    ViewData["Title"] = "Consulta LINQ";
}

<div class="panel">
    <p class="tituloForm">Ejemplo 1 · Consulta con LINQ (EF Core)</p>

    <form method="post">
        <div asp-validation-summary="All" class="text-danger mb-2"></div>
        <div class="row g-3">
            <div class="col-md-4">
                <label asp-for="Codigo" class="form-label">Codigo de cliente</label>
                <input asp-for="Codigo" class="form-control" placeholder="C001" />
                <span asp-validation-for="Codigo" class="text-danger"></span>
            </div>
            <div class="col-md-4">
                <label asp-for="FechaInicio" class="form-label">Desde</label>
                <input asp-for="FechaInicio" type="date" class="form-control" />
            </div>
            <div class="col-md-4">
                <label asp-for="FechaFin" class="form-label">Hasta</label>
                <input asp-for="FechaFin" type="date" class="form-control" />
            </div>
        </div>
        <button type="submit" class="btn btn-vl mt-3">Consultar</button>
    </form>

    @if (Model.Consultado)
    {
        <div class="resultado mt-3">
            Deuda actual del cliente (facturas pendientes):
            <strong>$/ @Model.Deuda.ToString("0.00")</strong>
        </div>
    }
}
```

10. Ejemplo 1 · Consulta LINQ (vista) (2 de 2)

Pages \ Consulta.cshtml (continuación)

```

</div>

@if (Model.Facturas.Count > 0)
{
    <table class="table tabla-vl mt-3">
        <thead>
            <tr><th>Nº Factura</th><th>Fecha</th><th>Estado</th><th class="text-end">Total</th>
        </tr>
        </thead>
        <tbody>
            @foreach (var f in Model.Facturas)
            {
                <tr>
                    <td>@f.NumFac</td>
                    <td>@f.FecFac.ToString("dd/MM/yyyy")</td>
                    <td>@f.EstadoTexto</td>
                    <td class="text-end">S/ @(f.Detalles.Sum(d => d.CanVen *
d.PreVen).ToString("0.00"))</td>
                </tr>
            }
        </tbody>
    </table>
}
else
{
    <div class="vl-nota mt-3">No se encontraron facturas para ese cliente en el rango indicado.
</div>
}

<div class="vl-nota">
    La pagina llama a la capa de datos (<strong>VentasRepositorio</strong>), que consulta con LINQ.
    La vista nunca habla directo con la base de datos.
</div>
</div>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

10. Ejemplo 1 · Consulta LINQ (código) (1 de 2)

Pages \ Consulta.cshtml.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using System.ComponentModel.DataAnnotations;
using Web_Sesion12.Datos;
using Web_Sesion12.Modelos;

namespace Web_Sesion12.Pages;

// Ejemplo 1: consultas con LINQ sobre EF Core (deuda + facturas entre fechas).
public class ConsultaModel : PageModel
{
    private readonly VentasRepositorio _repo;

    // La capa de datos llega por inyeccion
    public ConsultaModel(VentasRepositorio repo)
    {
        _repo = repo;
    }

    [BindProperty]
    [Required(ErrorMessage = "Ingrese el codigo del cliente")]
    public string Codigo { get; set; } = "";

    [BindProperty, DataType(DataType.Date)]
    public DateTime FechaInicio { get; set; } = new DateTime(2025, 1, 1);

    [BindProperty, DataType(DataType.Date)]
    public DateTime FechaFin { get; set; } = new DateTime(2025, 12, 31);

    public bool Consultado { get; set; }
    public decimal Deuda { get; set; }
    public List<Factura> Facturas { get; set; } = new();

    public void OnGet() { }
```


10. Ejemplo 1 · Consulta LINQ (código) (2 de 2)

Pages \ Consulta.cshtml.cs (continuación)

```
public async Task OnPostAsync()
{
    if (!ModelState.IsValid)
        return;

    // envio de datos a la capa de datos: se calcula la deuda y se listan las facturas
    Deuda = await _repo.CalcularDeudaClienteLinqAsync(Codigo);
    Facturas = await _repo.ListarFacturasLinqAsync(Codigo, FechaInicio, FechaFin);
    Consultado = true;
}
}
```

Pruebe con C001: el `OnPost` llama a la capa de datos, que calcula la deuda con LINQ y lista las facturas del rango.

11. Ejemplo 2 · Procedimientos (vista) (1 de 2)

Agregue la Razor Page **Procedimiento**:

Pages \ Procedimiento.cshtml

```
@page
@model ProcedimientoModel
@{
    ViewData["Title"] = "Procedimiento almacenado";
}

<div class="panel">
    <p class="tituloForm">Ejemplo 2 · Procedimiento almacenado (EF Core)</p>

    <form method="post">
        <div asp-validation-summary="All" class="text-danger mb-2"></div>
        <div class="row g-3">
            <div class="col-md-4">
                <label asp-for="Codigo" class="form-label">Codigo de cliente</label>
                <input asp-for="Codigo" class="form-control" placeholder="C001" />
                <span asp-validation-for="Codigo" class="text-danger"></span>
            </div>
            <div class="col-md-4">
                <label asp-for="FechaInicio" class="form-label">Desde</label>
                <input asp-for="FechaInicio" type="date" class="form-control" />
            </div>
            <div class="col-md-4">
                <label asp-for="FechaFin" class="form-label">Hasta</label>
                <input asp-for="FechaFin" type="date" class="form-control" />
            </div>
        </div>
        <button type="submit" class="btn btn-vl mt-3">Ejecutar procedimientos</button>
    </form>

    @if (Model.Consultado)
    {
        <div class="resultado mt-3">
            Deuda (vía SP <strong>usp_DeudaCliente</strong>, parámetro de salida):
            <strong>$/ @Model.Deuda.ToString("0.00")</strong>
        </div>
    }
}
```

11. Ejemplo 2 · Procedimientos (vista) (2 de 2)

Pages \ Procedimiento.cshtml (continuación)

```

</div>

@if (Model.Facturas.Count > 0)
{
    <p class="mt-3 mb-1 text-muted">Resultado del SP
<strong>usp_ListarFacturasClienteFechas</strong>:</p>
    <table class="table tabla-vl">
        <thead>
            <tr><th>Nº Factura</th><th>Fecha</th><th>Estado</th><th class="text-end">Total</th>
</tr>
        </thead>
        <tbody>
            @foreach (var f in Model.Facturas)
            {
                <tr>
                    <td>@f.Num_fac</td>
                    <td>@f.Fec_fac.ToString("dd/MM/yyyy")</td>
                    <td>@f.Estado</td>
                    <td class="text-end">S/ @f.Total.ToString("0.00")</td>
                </tr>
            }
        </tbody>
    </table>
}
else
{
    <div class="vl-nota mt-3">El procedimiento no devolvió facturas para ese rango.</div>
}

<div class="vl-nota">
    Mismo resultado que el Ejemplo 1, pero la lógica vive en la base de datos.
    EF Core ejecuta los SP con <strong>FromSql</strong> (filas) y un <strong>parámetro de
salida</strong> (escalar).
</div>
</div>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

11. Ejemplo 2 · Procedimientos (código) (1 de 2)

Pages \ Procedimiento.cshtml.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using System.ComponentModel.DataAnnotations;
using Web_Sesion12.Datos;
using Web_Sesion12.Modelos;

namespace Web_Sesion12.Pages;

// Ejemplo 2: ejecutar procedimientos almacenados desde EF Core.
public class ProcedimientoModel : PageModel
{
    private readonly VentasRepositorio _repo;

    public ProcedimientoModel(VentasRepositorio repo)
    {
        _repo = repo;
    }

    [BindProperty]
    [Required(ErrorMessage = "Ingrese el codigo del cliente")]
    public string Codigo { get; set; } = "";

    [BindProperty, DataType(DataType.Date)]
    public DateTime FechaInicio { get; set; } = new DateTime(2025, 1, 1);

    [BindProperty, DataType(DataType.Date)]
    public DateTime FechaFin { get; set; } = new DateTime(2025, 12, 31);

    public bool Consultado { get; set; }
    public decimal Deuda { get; set; }
    public List<FacturaResumen> Facturas { get; set; } = new();

    public void OnGet() { }
```

11. Ejemplo 2 · Procedimientos (código) (2 de 2)

Pages \ Procedimiento.cshtml.cs (continuación)

```
public async Task OnPostAsync()
{
    if (!ModelState.IsValid)
        return;

    // se invocan los procedimientos almacenados a traves de la capa de datos
    Deuda = await _repo.CalcularDeudaClienteSpAsync(Codigo);
    Facturas = await _repo.ListarFacturasSpAsync(Codigo, FechaInicio, FechaFin);
    Consultado = true;
}
}
```

Mismo resultado que el Ejemplo 1, pero la lógica vive en la base de datos. EF Core ejecuta los SP con `FromSql` y con un parámetro de salida.

12. Ejemplo 3 · AJAX (vista) (1 de 3)

Agregue la Razor Page **Ajax**:

Pages \ Ajax.cshtml

```
@page
@model AjaxModel
@{
    ViewData["Title"] = "AJAX moderno";
}

<div class="panel">
    <p class="tituloForm">Ejemplo 3 · AJAX moderno (fetch + EF Core)</p>

    <!-- No hay <form method=post>: todo se resuelve sin recargar la pagina -->
    <div class="row g-3 align-items-end">
        <div class="col-md-3">
            <label class="form-label">Codigo de cliente</label>
            <input id="codigo" class="form-control" placeholder="C001" />
        </div>
        <div class="col-md-3">
            <label class="form-label">Desde</label>
            <input id="desde" type="date" class="form-control" value="2025-01-01" />
        </div>
        <div class="col-md-3">
            <label class="form-label">Hasta</label>
            <input id="hasta" type="date" class="form-control" value="2025-12-31" />
        </div>
        <div class="col-md-3">
            <button id="btnBuscar" type="button" class="btn btn-vl w-100">Buscar (sin recargar)
        </button>
    </div>
</div>

<!-- Este panel se actualiza por AJAX; el resto de la pagina no se recarga -->
<div id="panelCliente" class="resultado mt-3" style="display:none">
    <div><strong id="razon"></strong></div>
    <div class="text-muted"><span id="ruc"></span> · <span id="dir"></span></div>
    <div class="mt-2">Deuda pendiente: <strong id="deuda">$/ 0.00</strong></div>
</div>
```

12. Ejemplo 3 · AJAX (vista) (2 de 3)

Pages \ Ajax.cshtml (continuación)

```

<div id="avisoVacio" class="vl-nota mt-3" style="display:none">No se encontró un cliente con ese
código.</div>

<table id="tablaFacturas" class="table tabla-vl mt-3" style="display:none">
  <thead><tr><th>N° Factura</th><th>Fecha</th><th>Estado</th><th class="text-end">Total</th></tr>
</thead>
  <tbody id="cuerpoFacturas"></tbody>
</table>

<hr class="my-4" />

<!-- Monto con "mascara": se formatea como soles al salir del campo (equivalente moderno del
MaskedEdit) -->
<div class="row g-3 align-items-end">
  <div class="col-md-4">
    <label class="form-label">Monto a registrar (demo de máscara)</label>
    <input id="monto" class="form-control" inputmode="decimal" placeholder="0.00" />
  </div>
  <div class="col-md-4">
    <button id="btnRegistrar" type="button" class="btn btn-outline-secondary w-100">Registrar
(pide confirmación)</button>
  </div>
</div>

<div class="vl-nota">
  El botón "Buscar" llama por <strong>fetch</strong> al handler
<strong>OnGetBuscarCliente</strong>,
  que consulta la BD vía EF Core y devuelve JSON. JavaScript actualiza solo el panel, sin
recargar.
</div>
</div>

@section Scripts {
  <script>
    // pide los datos del cliente al servidor (handler BuscarCliente) y refresca solo el panel
    async function buscarCliente() {
      const codigo = document.getElementById("codigo").value.trim();
      const desde = document.getElementById("desde").value;
      const hasta = document.getElementById("hasta").value;
      if (codigo === "") { alert("Ingrese un código de cliente."); return; }

      // llamada AJAX al handler de la pagina: ?handler=BuscarCliente
      const r = await fetch(`?handler=BuscarCliente&codigo=${encodeURIComponent(codigo)}`);
      const data = await r.json();

      const panel = document.getElementById("panelCliente");
      const aviso = document.getElementById("avisoVacio");

      if (!data.encontrado) {
        panel.style.display = "none";
        document.getElementById("tablaFacturas").style.display = "none";
        aviso.style.display = "block";
        return;
      }
    }
  </script>
}

```

12. Ejemplo 3 · AJAX (vista) (3 de 3)

Pages \ Ajax.cshtml (continuación)

```

        document.getElementById("dir").textContent = data.dirCli || "";
        // formatearSoles() viene del script externo site-demo.js
        document.getElementById("deuda").textContent = formatearSoles(data.deuda);
        panel.style.display = "block";

        // segunda llamada AJAX: trae las facturas del rango y arma la tabla
        await cargarFacturas(codigo, desde, hasta);
    }

    // pide las facturas del rango (handler Facturas) y llena la tabla sin recargar
    async function cargarFacturas(codigo, desde, hasta) {
        const r = await fetch(`?
handler=Facturas&codigo=${encodeURIComponent(codigo)}&ini=${desde}&fin=${hasta}`);
        const lista = await r.json();
        const cuerpo = document.getElementById("cuerpoFacturas");
        const tabla = document.getElementById("tablaFacturas");
        cuerpo.innerHTML = "";

        if (!lista || lista.length === 0) { tabla.style.display = "none"; return; }

        lista.forEach(f => {
            const fila = document.createElement("tr");
            fila.innerHTML = `<td>${f.numFac}</td><td>${f.fecha}</td><td>${f.estado}</td>` +
                `<td class="text-end">${formatearSoles(f.total)}</td>`;
            cuerpo.appendChild(fila);
        });
        tabla.style.display = "table";
    }

    // da formato de moneda al monto cuando el usuario sale del campo
    function aplicarMascaraMonto() {
        const campo = document.getElementById("monto");
        const valor = aNumero(campo.value);
        campo.value = valor.toFixed(2);
    }

    // pide confirmacion antes de "registrar" (equivalente moderno del mensaje de confirmacion)
    function registrar() {
        const monto = document.getElementById("monto").value;
        if (confirm("¿Seguro de registrar el monto " + formatearSoles(aNumero(monto)) + "?")) {
            alert("Registrado (demostración).");
        }
    }

    document.getElementById("btnBuscar").addEventListener("click", buscarCliente);
    document.getElementById("monto").addEventListener("blur", aplicarMascaraMonto);
    document.getElementById("btnRegistrar").addEventListener("click", registrar);
</script>
}

```


12. Ejemplo 3 · AJAX (handlers) (1 de 2)

Pages \ Ajax.cshtml.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Web_Sesion12.Datos;

namespace Web_Sesion12.Pages;

// Ejemplo 3: AJAX moderno. Los handlers OnGet... devuelven JSON; el navegador
// los llama con fetch y actualiza solo una parte de la pagina, sin recargar.
public class AjaxModel : PageModel
{
    private readonly VentasRepositorio _repo;

    public AjaxModel(VentasRepositorio repo)
    {
        _repo = repo;
    }

    public void OnGet() { }

    // handler AJAX: devuelve el cliente y su deuda en JSON, sin recargar la pagina.
    // Se invoca desde JavaScript como ?handler=BuscarCliente
    public async Task<IActionResult> OnGetBuscarClienteAsync(string codigo)
    {
        var cliente = await _repo.BuscarClienteAsync(codigo);
        if (cliente == null)
            return new JsonResult(new { encontrado = false });

        decimal deuda = await _repo.CalcularDeudaClienteLinqAsync(codigo);

        return new JsonResult(new
        {
            encontrado = true,
            cliente.CodCli,
            cliente.RazSocCli,
        });
    }
}
```

12. Ejemplo 3 · AJAX (handlers) (2 de 2)

Pages \ Ajax.cshtml.cs (continuación)

```
        cliente.RucCli,
        cliente.DirCli,
        deuda
    });
}

// handler AJAX: devuelve las facturas del rango como una lista JSON.
// Se invoca desde JavaScript como ?handler=Facturas
public async Task<IActionResult> OnGetFacturasAsync(string codigo, DateTime ini, DateTime fin)
{
    var facturas = await _repo.ListarFacturasLinqAsync(codigo, ini, fin);

    // se devuelve solo lo necesario para la tabla (no toda la entidad)
    var datos = facturas.Select(f => new
    {
        numFac = f.NumFac,
        fecha = f.FecFac.ToString("dd/MM/yyyy"),
        estado = f.EstadoTexto,
        total = f.Detalles.Sum(d => d.CanVen * d.PreVen)
    });

    return new JsonResult(datos);
}
```

Regla de oro: el fetch llama al handler, el handler usa la capa de datos (EF Core) y devuelve solo JSON. La vista nunca toca la base de datos.

13. Probar la aplicación

Paso 1 Verifique que ejecutó `VentasLeon.sql` y que la cadena de `appsettings.json` apunta a su servidor.

Paso 2 Presione **F5**. Se abre la página de inicio con los tres ejemplos.

Paso 3 **1 · Consulta LINQ**: escriba `C001` y consulte. Verá la deuda y las facturas.

Paso 4 **2 · Procedimiento**: repita con `C001`. Mismo resultado, vía procedimientos almacenados.

Paso 5 **3 · AJAX**: escriba `C001` y pulse "Buscar". El panel se actualiza sin recargar la página.

Si algo falla: lo más común es que la BD no exista o que la cadena de conexión no coincida con su instancia (use `Server=.\SQLEXPRESS` si esa es la suya).

Con esto quedan cubiertos los tres pilares de la sesión: consultas con LINQ, procedimientos almacenados y AJAX, todo sobre Entity Framework Core.